

LangChain Fundamentals

Models, Messages, and Application Architecture

A practical guide for professionals building applications with language models — from first concepts to a complete request-response workflow.



Agenda

What We'll Cover

01

What Is LangChain?

Definition, role, and where it fits in the AI stack

02

The Ecosystem

LangChain, LangGraph, and LangSmith — and how they relate

03

Core Building Blocks

Models, messages, templates, output handling, documents

04

Models in Depth

Interface, configuration, invocation, and responses

05

Putting It Together

Architecture, real workflows, troubleshooting, and limitations

What Is LangChain?

LangChain is an **open-source framework** for building applications that use large language models (LLMs). It provides a structured way to connect models to prompts, data, memory, and output processing — without having to wire each piece together from scratch.

What LangChain Provides

- A common interface across many model providers
- Reusable components for prompts, parsing, and retrieval
- Composable chains that link steps into workflows

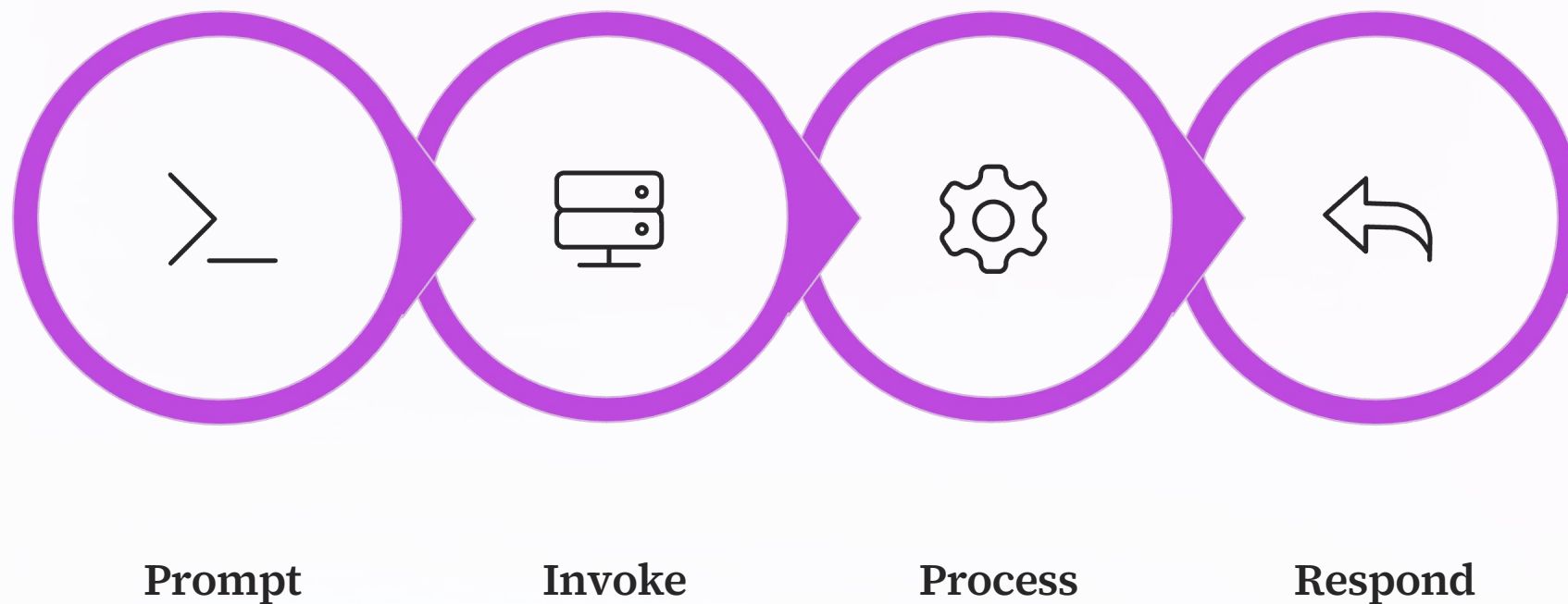
What LangChain Is Not

- Not a model itself — it calls models on your behalf
- Not a silver bullet for hallucinations or missing knowledge
- Not required for every LLM task — simpler APIs work for simple tasks



Orchestration: The Central Idea

Orchestration means coordinating multiple components — prompts, model calls, data sources, and output processing — so they work together as a cohesive application. LangChain is an orchestration framework: it manages the flow of information between these parts, so you write application logic instead of integration glue.



Each arrow above represents a hand-off that LangChain manages on your behalf. Your application code describes *what* to do; LangChain handles *how* the pieces connect.

Direct API Call vs. Orchestrated Application

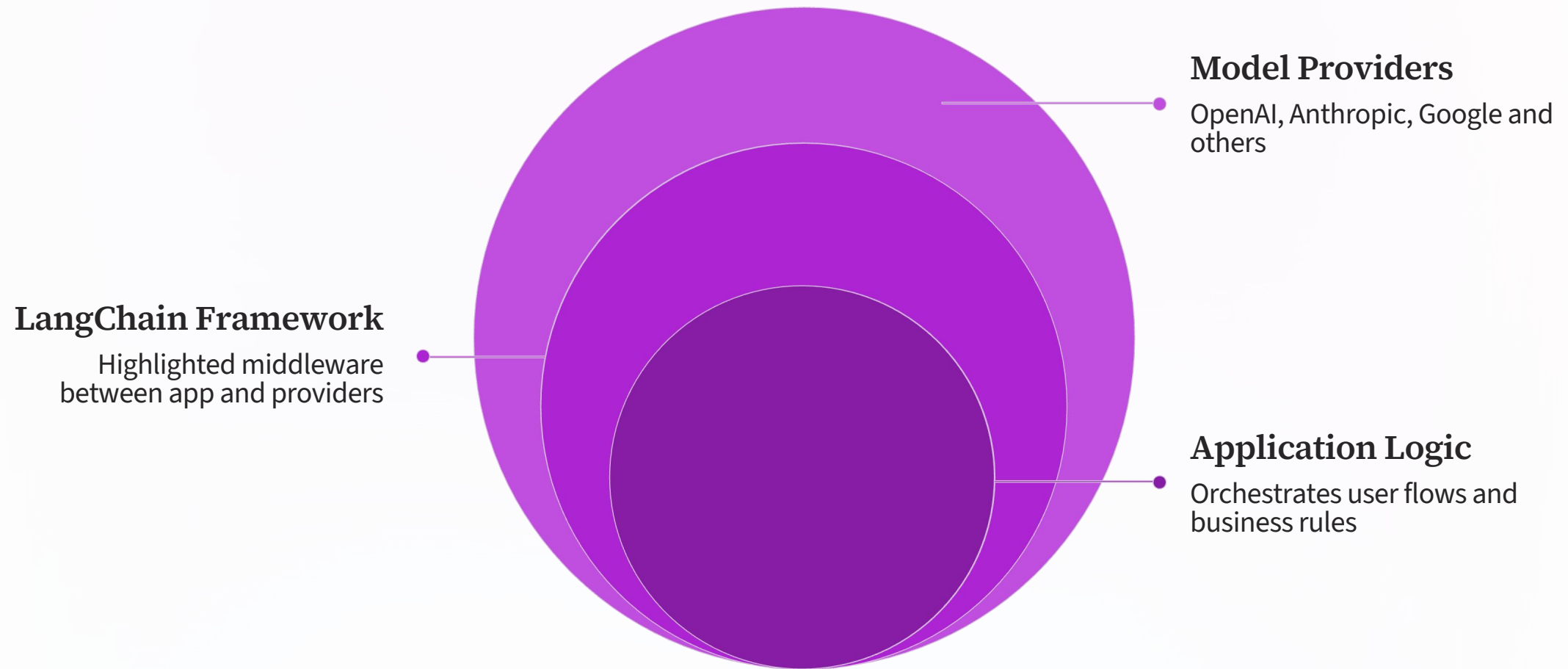
Both approaches work. The right choice depends on what your application actually needs.

Dimension	Direct API Call	LangChain Orchestration
Setup complexity	Minimal — one HTTP request	Higher — components to configure
Reusability	Manual — you rebuild each time	Built-in — templates and chains reuse
Provider switching	Requires code changes per provider	Common interface reduces changes
Multi-step workflows	You manage each step manually	Chains compose steps declaratively
Observability	You add your own logging	LangSmith integrates directly
Best fit	Single-shot, one-off tasks	Multi-step, production applications

 A customer-support bot that routes, retrieves policy docs, and formats replies benefits from orchestration. A one-off summarization script probably does not.

Where LangChain Sits in Your Stack

LangChain occupies the layer **between your application logic and the model providers**. Your code talks to LangChain; LangChain talks to OpenAI, Anthropic, Google, or any other provider you configure. This separation means you can change providers without rewriting your application.



The middle layer also normalizes inputs and outputs, so a response from Claude looks structurally similar to one from GPT-4 in your application code — even though the underlying models differ significantly.

The LangChain Ecosystem

Three distinct tools share the LangChain name. Understanding their roles prevents confusion as you explore the documentation.

LangChain (Core)

The primary framework. Provides the abstractions for models, messages, prompt templates, output parsers, document loaders, and retrievers. This is where almost everything in this presentation lives.

LangGraph

An extension for building **stateful, multi-step agent workflows** as explicit graphs. Useful when your application needs to loop, branch, or maintain complex state across many model calls. Out of scope for today.

LangSmith

A **developer observability platform**. Logs every prompt, response, latency, and token count for debugging and evaluation. Works alongside LangChain but is a separate product, not a requirement.

 Think of LangChain as the engine, LangGraph as the transmission for complex routes, and LangSmith as the dashboard that shows you what's happening under the hood.

Core Building Blocks

LangChain's power comes from a small set of composable primitives. Learn these and you can build almost anything.



The Six Core Primitives

Chat Models

The interface to a language model. Accepts a list of messages and returns a response message.

Prompt Templates

Reusable prompt patterns with variable slots. Fill in the blanks at runtime without rewriting prompts.

Documents

A standard container for text plus metadata (source, page, date) loaded from files, databases, or APIs.

Messages

Typed conversation turns — System, Human/User, and AI/Assistant — that structure the prompt.

Output Parsers

Transform raw model text into structured data — JSON, lists, typed objects — your application can use.

Retrievers

Find and return relevant Documents given a query. The foundation of retrieval-augmented generation (RAG) — covered in detail later.

Provider Integrations and the Common Interface

LangChain ships with integrations for dozens of model providers. Each integration wraps the provider's native SDK but exposes a **common interface** — the same `invoke()` call works whether you're talking to OpenAI, Anthropic, Cohere, or a locally running model.

Why a Common Interface Matters

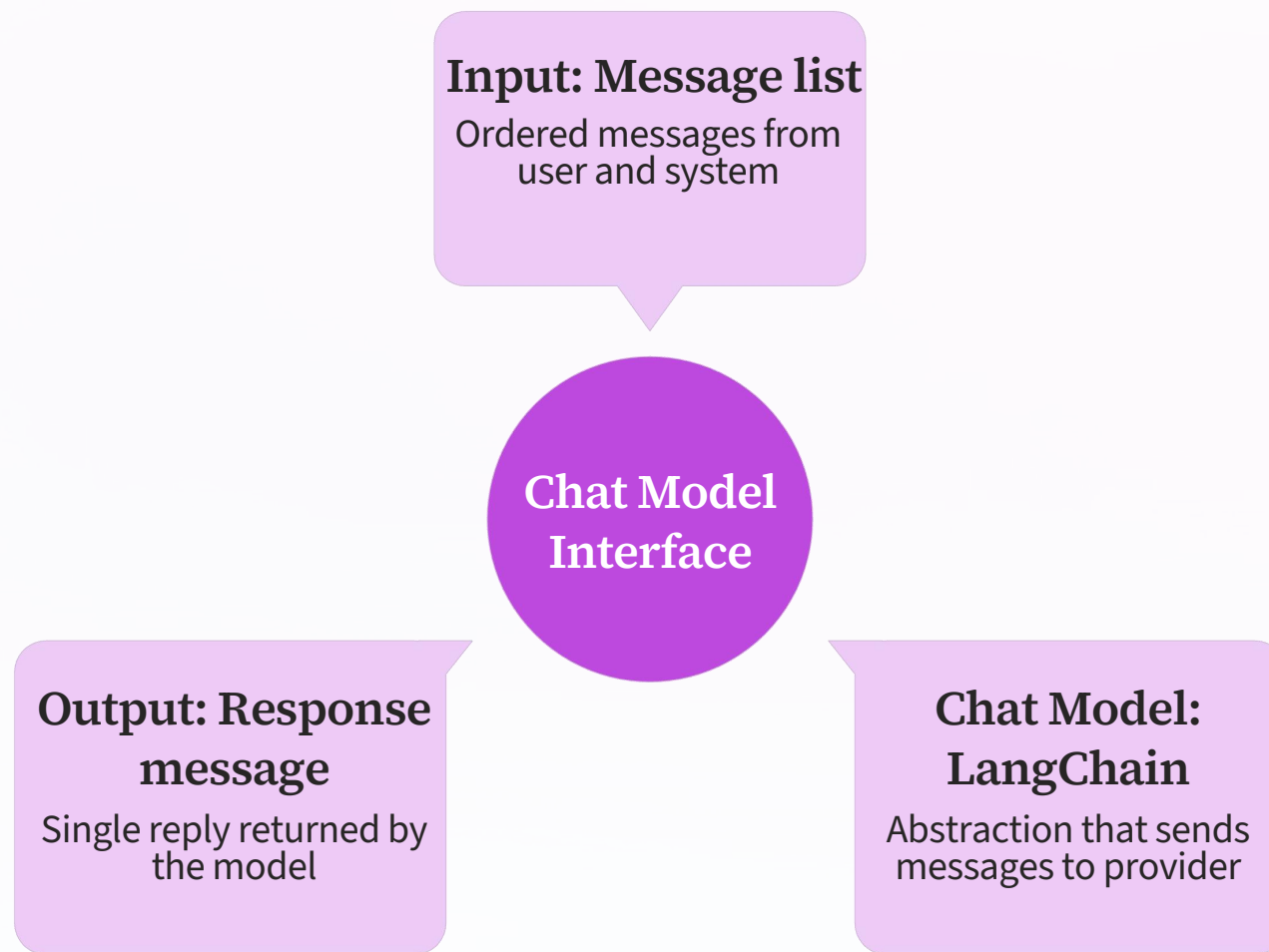
- Switch providers by changing one configuration line
- Test prompts across models without rewriting your chain
- Reduce provider-specific boilerplate in application code
- Standardize error handling across different SDKs

Important Caveat

A common interface does **not** mean identical behavior. Models differ in context window size, supported parameters, safety filtering, function-calling capabilities, and output style. Always consult the provider's own documentation for capabilities.

The Chat Model Interface

In LangChain, a **chat model** is not a direct window into a model provider's raw API. It is a LangChain abstraction that accepts a **list of messages**, sends them to the provider, and returns a single **response message**. This three-part contract is consistent across every supported provider.



- **Input:** An ordered list of typed message objects representing the conversation so far
- **Model:** The LangChain chat model object, configured with provider credentials and settings
- **Output:** An AI message object containing generated text and metadata

System, User, and Assistant Messages

Every conversation in LangChain is built from three message roles. Each has a distinct purpose and position in the conversation list.

System Message

Sets the model's persona, constraints, and behavioral rules. Written by the developer. Processed before the conversation begins.

Example: "You are a helpful customer support agent for Acme Corp. Always respond politely and cite the policy document when relevant."

Human / User Message

Represents input from the end user. This is the question, request, or statement the model should respond to. *Example: "My order has not arrived. What should I do?"*

AI / Assistant Message

Represents a prior response from the model. When included in the message list, it gives the model a record of what it already said — enabling multi-turn conversation. *Example: "I'm sorry to hear that. Can you share your order number?"*

An Annotated Conversation

Below is a realistic three-turn exchange, annotated by role. Notice how each message builds on the last, and how the system message shapes the assistant's tone throughout.

① **System** — Developer-authored rules

"You are a polite customer-support assistant for Acme Shipping. If a customer asks about a lost package, ask for their order number before escalating."

② **Human** — User's first message

"Hi, my package never showed up and I'm really frustrated."

③ **AI** — Model's first response (now part of history)

"I completely understand your frustration and I'm here to help. Could you please share your order number so I can look into this right away?"

④ **Human** — User follows up

"It's ACM-98234."

 All four messages would be passed as a list to the model on the next invocation — the system instruction remains, and the model sees the full context.

Structured Messages vs. One Big Text Prompt

Older model APIs accepted a single text string. Modern chat models — and LangChain — use **structured message lists**. The difference has real consequences for reliability and behavior.

Dimension	Single Text Prompt	Structured Messages
Role clarity	Model infers who said what	Each turn is explicitly labeled
Instruction separation	System rules mixed with user text	System message is distinct and protected
Multi-turn support	Requires manual formatting	Each turn is a separate object
Prompt injection risk	Higher — user text can override instructions	Lower — roles are structurally separated
Debugging	Hard to isolate which part caused an issue	Each role is inspectable independently

Conversation History and State Management

Language models are **stateless**. Each invocation is independent — the model has no memory of previous calls. The illusion of a continuous conversation is maintained entirely by the **application**, which must store and re-send the message history on every turn.

Application Responsibilities

- Store every message (Human and AI) after each turn
- Decide how many historical turns to include
- Trim or summarize history when the context limit approaches
- Handle session boundaries (e.g., new user, new conversation)

LangChain's Role

LangChain provides **memory** utilities and **chat history** abstractions that simplify storage and retrieval of conversation history — but the application developer still decides the strategy. Memory is a tool, not an automatic feature.

 Passing too much history consumes tokens and can exceed the model's context window. Passing too little causes the model to lose important context. Managing this tradeoff is a core design decision.

Model Configuration

Before invoking a model, you configure it. Configuration settings fall into two distinct categories — confusing them is a common source of bugs.

Generation Settings

Control the model's output behavior:

- **Model name** — which model to use (e.g., `gpt-4o`, `claude-3-5-sonnet`)
- **Temperature** — 0 for deterministic, higher for creative variety
- **Max tokens** — caps the length of the generated response
- **Stop sequences** — tokens that signal the model to stop generating

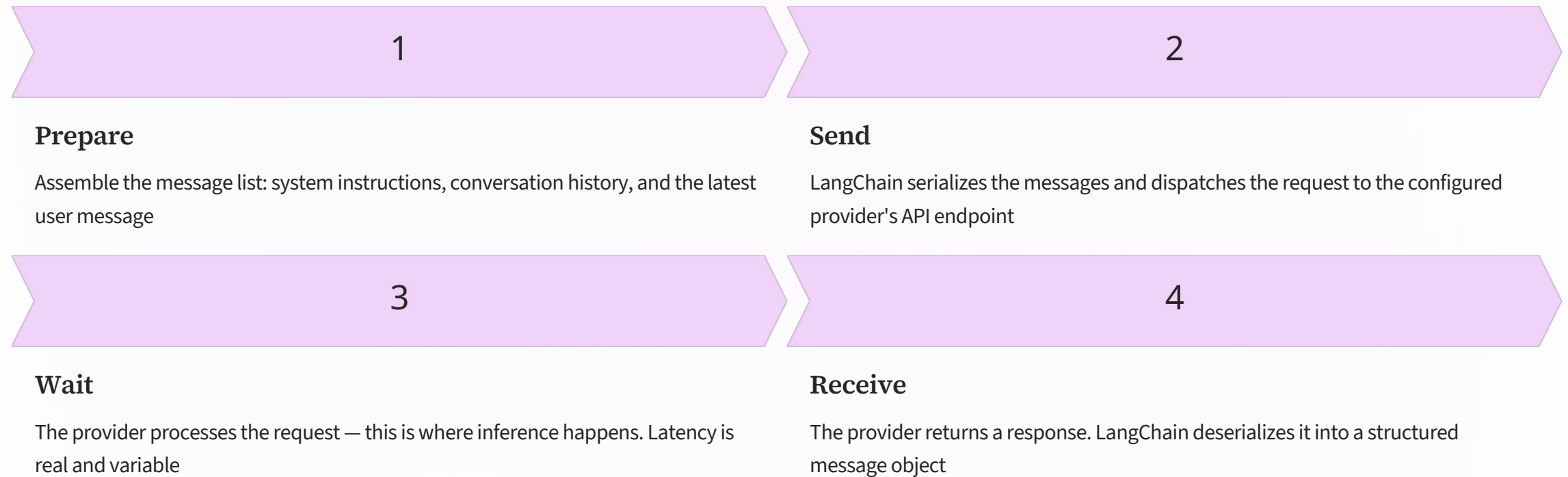
Reliability Settings

Control how your application behaves around the model call:

- **Timeout** — how long to wait before giving up on a slow response
- **Max retries** — how many times to retry on transient network or provider errors
- **Callbacks** — hooks that fire before, during, and after invocation for logging

What "Invoking" a Model Means

In LangChain, **invocation** is the act of sending the prepared message list to the model and waiting for a response. It is a network call — it takes time, consumes tokens, and can fail.



i LangChain also supports **streaming** — receiving tokens as they are generated rather than waiting for the complete response. Streaming is especially useful for chat interfaces where users expect to see text appear in real time.

What a Response Contains

A model invocation returns an **AI message object**. Its contents vary by provider, but most responses share a consistent set of fields.

Core Fields (Nearly Universal)

- **content** — the generated text string
- **role** — always `assistant` for model responses
- **response_metadata** — provider-specific metadata (model name, finish reason, timestamps)

Usage Information (When Supported)

- **input_tokens** — tokens consumed by the prompt
- **output_tokens** — tokens in the generated reply
- **total_tokens** — sum, used for cost estimation

Finish Reason

Tells you *why* the model stopped generating:

- `stop` — natural end of response
- `length` — hit the `max_tokens` limit
- `content_filter` — blocked by safety layer
- `tool_calls` — model wants to call a function

Always check finish reason before using the output — a `length` stop means the response is incomplete.

Receiving, Extracting, and Validating

Getting a response is not the same as having a usable result. Three distinct steps follow invocation — and skipping any of them is a common source of application bugs.



1. Receive the Response

The AI message object arrives. At this point you have raw data. Check that the call succeeded (no exception, valid object) and inspect the finish reason before proceeding.



2. Extract the Text

Pull the `content` field from the message. If you asked for structured output (JSON, a list, a typed object), apply the appropriate output parser here. Parsing can fail if the model deviated from the requested format.

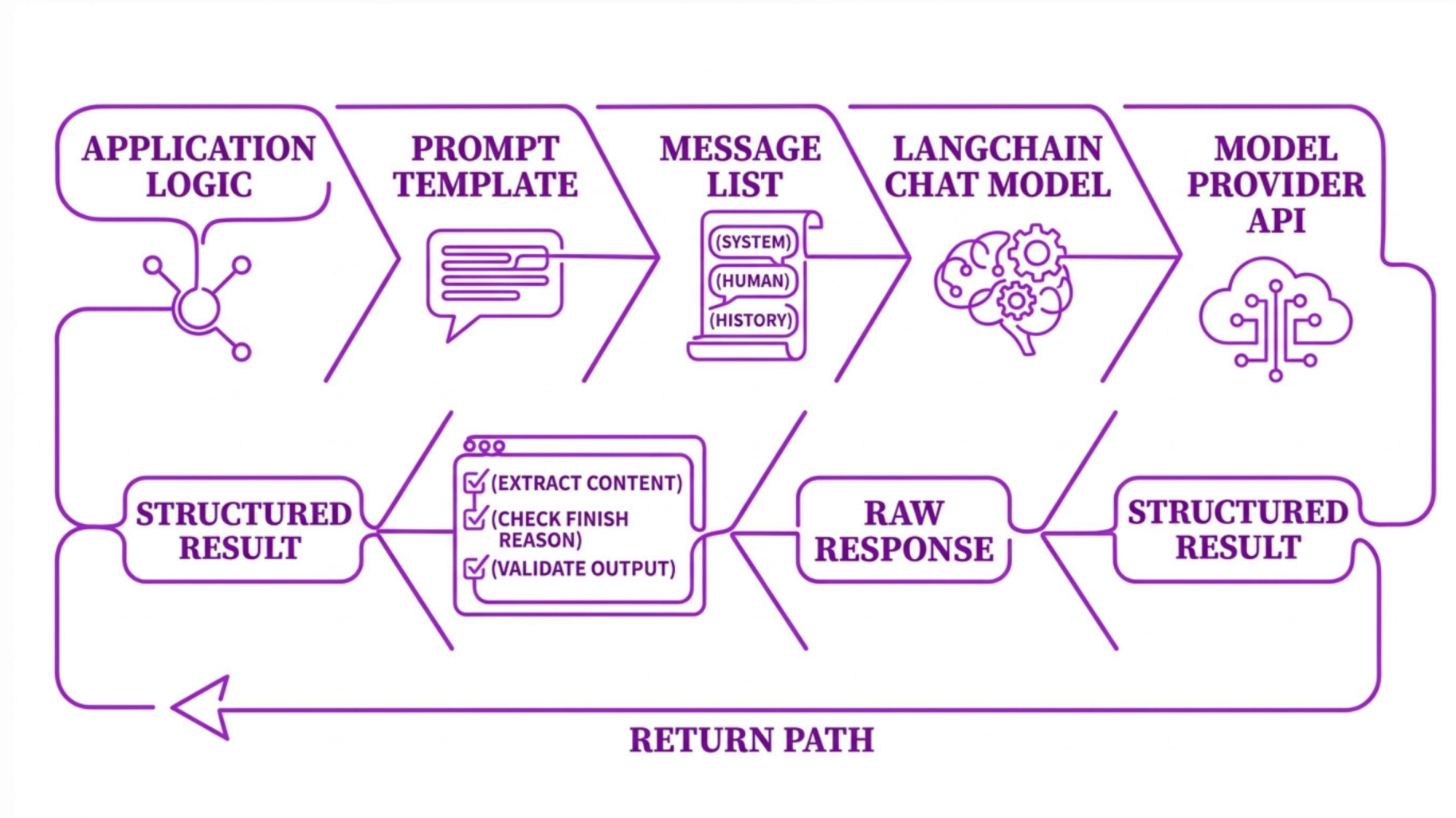


3. Validate Suitability

Confirm the output meets your application's requirements: Is it the right format? Is it within acceptable length? Does it contain required fields? Validation logic belongs in your application, not in LangChain itself.

The Complete Application Architecture

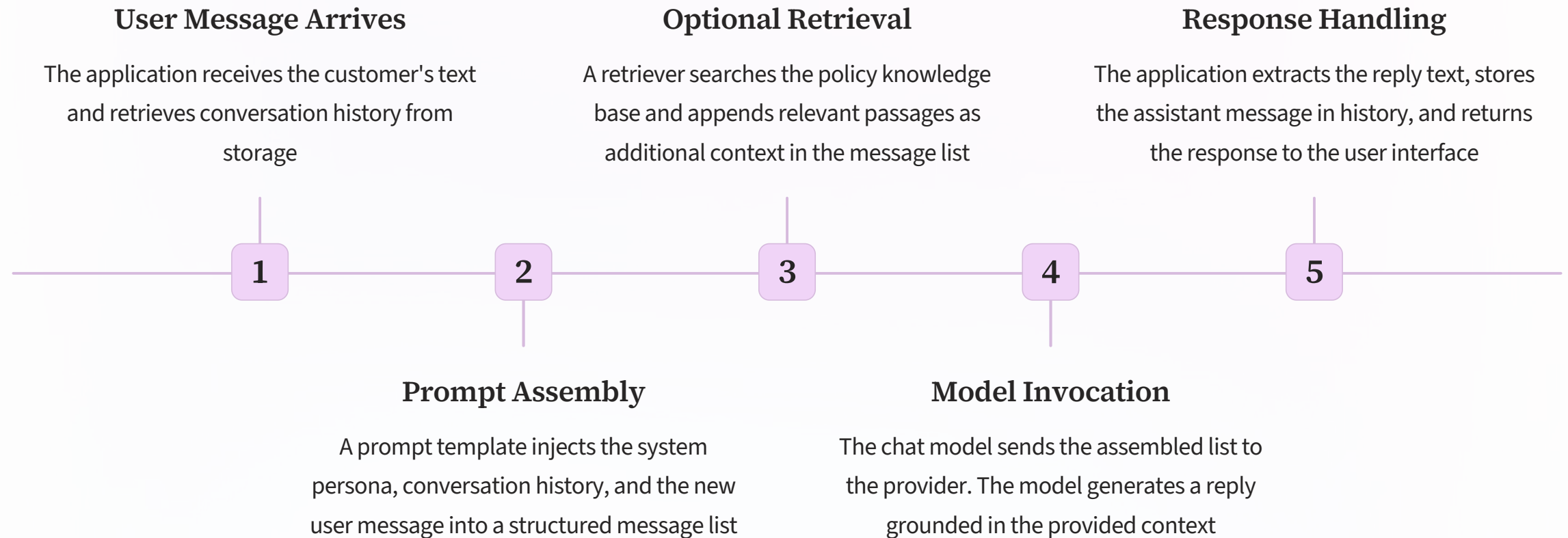
Every component covered so far connects into a single end-to-end flow. This diagram is the central mental model for working with LangChain.



The **outbound path** (left to right) assembles and delivers the prompt. The **inbound path** (right to left) extracts, validates, and returns the result. Your application logic sits at both ends.

Real Workflow: Customer-Support Response

A customer-support assistant receives a user's message, looks up relevant policy or order information, and generates a helpful reply. Here is how the LangChain components map to each step.



Real Workflow: Document Summarization

An internal knowledge assistant must summarize long documents — policy manuals, meeting transcripts, research reports. The flow differs from the conversational pattern in important ways.

1 Load and Split

A **document loader** reads the source file. A **text splitter** divides it into chunks that fit within the context window.

2 Summarize Per Chunk

Each chunk is passed to the model with a summarization prompt. The model returns a partial summary for each chunk.

3 Combine Summaries

Partial summaries are assembled and passed to a final model call that produces the consolidated document summary.

Why This Pattern Exists

A 50-page policy document almost certainly exceeds any model's context window. The map-reduce pattern — summarize chunks, then summarize the summaries — is LangChain's answer to documents that are too large to process in one call.

LangChain provides built-in chain types for this pattern, including **map-reduce** and **refine** strategies.

Common Troubleshooting Patterns

Most LangChain issues fall into one of six predictable categories. Recognizing the pattern speeds up diagnosis.

Pattern	Symptom	What to Check
Incorrect message roles	Model ignores instructions or responds unexpectedly	Verify system vs. human vs. AI assignment in message list
Missing context	Model doesn't know what was said earlier	Confirm history is loaded and passed on each invocation
Unsupported settings	Parameter silently ignored or API error returned	Check provider docs — not all parameters are universal
Malformed output	Parser fails; response is not valid JSON or expected format	Inspect raw content; tighten format instructions in prompt
Provider errors	Rate limit, timeout, or authentication failure	Check API key, quota limits, retry configuration
Context limit exceeded	Request rejected with "context length" error	Trim history, reduce document chunk size, upgrade model tier

What LangChain Does Not Solve

LangChain is a powerful framework, but it does not eliminate the fundamental limitations of language models. Being clear-eyed about this prevents overconfident deployments.

Hallucinations

Language models can generate plausible-sounding but factually incorrect content. LangChain provides no automatic factuality check. You must design validation, retrieval grounding, and human-review steps into your application.

Missing Business Knowledge

Models are trained on public data. They do not know your company's products, internal policies, pricing, or procedures unless that information is injected into the prompt — via templates, retrieval, or fine-tuning.

Deterministic Behavior

Even at temperature 0, model outputs are not guaranteed to be identical across providers, model versions, or time. If your application requires exact reproducibility, LLMs require careful design to meet that standard.

Cost and Latency

Every invocation consumes tokens and takes time. LangChain makes invocation easier, but does not reduce the per-token cost or the network round-trip. Budget and latency planning remain your responsibility.

Concept Recap and What Comes Next

LangChain

An orchestration framework that connects models, prompts, data, and output processing

Messages

System, Human, and AI roles structure every conversation sent to a chat model

Invocation

Prepare → Send → Wait → Receive. A real network call with real latency and cost

Responses

Receive, extract, and validate — three distinct steps before a result is usable

Limitations

LangChain does not eliminate hallucinations, inject business knowledge, or manage cost

Coming up next: [Prompt Templates and Reusable Prompting](#) — how to parameterize prompts for consistent, maintainable applications — followed by an introduction to [Retrieval-Augmented Generation \(RAG\)](#) for grounding responses in your organization's own knowledge.

References

All concepts in this presentation are grounded in official LangChain documentation and related sources. Verify details against current docs, as the framework evolves rapidly.

LangChain Official Documentation

python.langchain.com/docs/introduction — Conceptual guides, how-to articles, and API reference for LangChain (Python)

LangChain Messages Concept Guide

python.langchain.com/docs/concepts/messages — System, Human, AI, and tool message definitions and usage

LangGraph Documentation

langchain-ai.github.io/langgraph — Documentation for building stateful, graph-based agent workflows with LangGraph

LangChain Chat Models Concept Guide

python.langchain.com/docs/concepts/chat_models — Detailed explanation of the chat model interface, message types, and invocation

LangSmith Documentation

docs.smith.langchain.com — Official documentation for the LangSmith observability and evaluation platform

Provider Integration Index

python.langchain.com/docs/integrations/chat — Full list of supported chat model provider integrations and configuration options